

TASK 1 — Machine Learning Fundamentals.

1. What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every situation.

Instead of writing rules for every possible situation, we provide the computer with data. The ML algorithm studies the data, identifies patterns, and uses those patterns to make predictions on new data.

Example: A student performance model can learn from study hours, attendance, previous marks, and assignment scores to predict whether a student is likely to Pass or Fail.

2. AI vs Machine Learning vs Deep Learning



2. AI vs Machine Learning vs Deep Learning

Concept	Meaning	Example
Artificial Intelligence (AI)	Broad field of making machines perform tasks requiring human-like intelligence	Chatbots, self-driving cars
Machine Learning (ML)	A subset of AI where systems learn patterns from data	Spam detection
Deep Learning (DL)	A subset of ML using neural networks with multiple layers	Face recognition

3. What is a Dataset?

A dataset is a structured collection of information used by a machine learning algorithm.

A dataset normally contains rows and columns.

For example:

Study Hours	Attendance	Previous Marks	Result
6	85	72	Pass
2	60	45	Fail
8	90	88	Pass

4. Features and Labels

Features

Features are the input variables used by the ML model to make predictions.

For example:

Study Hours Attendance Previous Marks Assignment Score Label / Target

The label or target is the output that we want the model to predict.

Example:

Pass / Fail

Therefore:

Features → Input → Model → Target → Output

5. Training Data and Testing Data

A dataset is generally divided into two parts:

Training Data: Training data is used to teach the machine learning model. The model learns patterns and relationships from this data.

Testing Data: Testing data is used to evaluate how well the trained model performs on previously unseen data.

For example, we can use: 80% → Training Data 20% → Testing Data

6. Supervised Learning

Supervised Learning is a type of machine learning where the model learns from data that already contains the correct answers or labels.

Example:

Study Hours	Attendance	Result
6	85	Pass
2	60	Fail

The model learns the relationship between the input features and the known result.

Examples: Student Pass/Fail prediction Email Spam/Not Spam House price prediction Disease prediction

7. Unsupervised Learning

Unsupervised Learning works with data that does not have predefined labels.

The model attempts to discover hidden patterns or groups within the data.

Example:

A shopping company can group customers according to their purchasing behavior.

Possible groups:

High-value customers Regular customers Occasional customers

Common unsupervised learning algorithms include:

K-Means Clustering DBSCAN Hierarchical Clustering

8. Classification vs Regression

Classification: Classification predicts a category or class.

Examples: Pass / Fail Spam / Not Spam Disease / No Disease Cat / Dog

Regression: Regression predicts a continuous numerical value.

Examples: House price Salary Temperature Sales revenue

Classification	Regression
Predicts categories	Predicts numerical values
Pass/Fail	House price
Spam/Not Spam	Salary
Logistic Regression	Linear Regression



9. Real-World Applications of Machine Learning

Machine Learning is used in many industries.

Healthcare

ML can help predict diseases and analyze medical images.

Banking

Banks use ML for fraud detection and credit risk analysis.

E-commerce

Companies use ML for product recommendations and customer analysis.

Education

ML can predict student performance and identify students who may need additional support.

Transportation

ML is used in traffic prediction, route optimization, and autonomous vehicles.

Entertainment

Streaming platforms use ML to recommend movies, music, and videos.

Cybersecurity

ML can identify suspicious activities and potential cyber attacks.

Conclusion:-

Machine Learning allows computers to learn from data and make predictions or decisions. Understanding datasets, features, labels, training data, testing data, supervised learning, and unsupervised learning provides a strong foundation for ML. Machine Learning is widely used in healthcare, banking, education, e-commerce, transportation, and many other fields. By preparing data properly and selecting an appropriate algorithm, useful predictions can be made from real-world data.

✓ TASK 2 — Dataset Preparation.



For this task, we will use the Iris dataset.

The Iris dataset contains information about different iris flowers.

Features:

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

Target:----Species

There are three classes:

1. Setosa,
2. Versicolor,
3. Virginica

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
```

```
iris = load_iris()

df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["target"] = iris.target

df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.shape
```

```
(150, 5)
```

```
df.dtypes
```

	0
sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
target	int64

```
dtype: object
```

```
df.isnull().sum()
```

	0
sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
target	0

```
dtype: int64
```

```
df = df.dropna()
```

```
X = df.drop("target", axis=1)  
y = df["target"]
```

```
print("Features:")
print(X.head())
```

```
print("\nTarget:")
print(y.head())
```

```
Features:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2
```

```
Target:
```

```
0    0
1    0
2    0
3    0
4    0
```

```
Name: target, dtype: int64
```



✓ TASK 3 — Train-Test Split & Data Preprocessing.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

We use 80% of the data for training so that the model has enough examples to learn patterns.

The remaining 20% is used for testing to check whether the model can make accurate predictions on data it has not seen before.

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (120, 4)
Testing data: (30, 4)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

```
print('First 5 rows of X_train_scaled:')
print(X_train_scaled[:5])
```

```
First 5 rows of X_train_scaled:
[[-1.72156775 -0.33210111 -1.34572231 -1.32327558]
```

```
[-1.12449223 -1.22765467  0.41450518  0.6517626 ]  
[ 1.14439475 -0.5559895   0.58484978  0.25675496]  
[-1.12449223  0.11567567 -1.28894078 -1.45494479]  
[-0.40800161 -1.22765467  0.13059752  0.12508575]]
```

Dataset



Data Cleaning



Features (X) + Target (y)



Train-Test Split



Training Data



Model Learning



Testing Data



Prediction



Evaluation



✓ TASK 4 — Build Your First Machine Learning Model

Decision Tree Classifier

A Decision Tree is a supervised machine learning algorithm used for classification and regression.

```
import pandas as pd  
  
from sklearn.datasets import load_iris  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier
```

```

from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

# Create DataFrame
X = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Create model
model = DecisionTreeClassifier(random_state=42)

# Train model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Display predictions
print("Actual values:")
print(y_test)

print("\nPredicted values:")
print(y_pred)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")

```

```

Actual values:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]

Predicted values:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 1 2 2 1 0 2 0]

Model Accuracy: 0.9333333333333333
Accuracy Percentage: 93.33333333333333 %

```

How does the Decision Tree work?

A Decision Tree makes decisions using a series of questions about the input features.

Iris Dataset



↓

Select X and y

↓

Train-Test Split

↓

Decision Tree

↓

Model Training

↓

Prediction

↓

Accuracy



✓ 🎓 TASK 5 — MINI PROJECT

Student Performance Prediction

Objective

The objective of this project is to build a Machine Learning classification model that predicts whether a student is likely to PASS or FAIL based on:

Study Hours Attendance Previous Marks Assignment Score

```
import io
import pandas as pd

csv_data = """
Study_Hours,Attendance,Previous_Marks,Assignment_Score,Result
8,95,88,92,Pass
6,85,72,80,Pass
7,90,80,85,Pass
3,65,50,55,Fail
2,60,45,50,Fail
9,96,92,95,Pass
5,78,65,70,Pass
4,70,55,60,Fail
10,98,95,96,Pass
```

```

1,55,40,45,Fail
6,88,75,82,Pass
3,62,48,52,Fail
8,92,85,90,Pass
5,75,60,68,Pass
2,58,42,48,Fail
7,86,78,84,Pass
4,68,52,58,Fail
9,94,90,93,Pass
6,82,70,76,Pass
3,64,46,54,Fail
8,89,82,87,Pass
2,57,44,46,Fail
5,80,68,72,Pass
4,72,58,62,Fail
10,97,94,98,Pass
7,91,81,86,Pass
1,52,35,40,Fail
6,84,73,79,Pass
3,66,49,56,Fail
9,93,89,91,Pass
"""

```

```

df = pd.read_csv(io.StringIO(csv_data))
df.head()

```

	Study_Hours	Attendance	Previous_Marks	Assignment_Score	Result
0	8	95	88	92	Pass
1	6	85	72	80	Pass
2	7	90	80	85	Pass
3	3	65	50	55	Fail
4	2	60	45	50	Fail

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```

print("Shape:")
print(df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

```

```

Shape:
(30, 5)

Data Types:
Study_Hours      int64
Attendance       int64
Previous_Marks   int64
Assignment_Score int64
Result           object
dtype: object

Missing Values:
Study_Hours      0
Attendance       0

```

```
Previous_Marks    0
Assignment_Score  0
Result            0
dtype: int64
```

```
df = df.dropna()

print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
Study_Hours    0
Attendance     0
Previous_Marks 0
Assignment_Score 0
Result        0
dtype: int64
```

```
df = df.dropna()

print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
Study_Hours    0
Attendance     0
Previous_Marks 0
Assignment_Score 0
Result        0
dtype: int64
```

```
df["Result"] = df["Result"].map({
    "Fail": 0,
    "Pass": 1
})
```

```
X = df[
    [
        "Study_Hours",
        "Attendance",
        "Previous_Marks",
        "Assignment_Score"
    ]
]

y = df["Result"]
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```



Training samples: 24
Testing samples: 6

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    random_state=42,
    max_depth=4
)

model.fit(X_train, y_train)
```

▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier(max_depth=4, random_state=42)

```
y_pred = model.predict(X_test)

print("Actual values:")
print(y_test.values)

print("\nPredicted values:")
print(y_pred)
```

Actual values:
[0 1 1 1 1 0]

Predicted values:
[0 1 1 1 1 0]

```
y_pred = model.predict(X_test)

print("Actual values:")
print(y_test.values)

print("\nPredicted values:")
print(y_pred)
```

Actual values:
[0 1 1 1 1 0]

Predicted values:
[0 1 1 1 1 0]

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
```

Model Accuracy: 1.0
Accuracy Percentage: 100.0 %

```
new_students = pd.DataFrame({
    "Study_Hours": [6, 2, 8],
    "Attendance": [85, 60, 92],
    "Previous_Marks": [72, 45, 85],
    "Assignment_Score": [80, 50, 90]
```

```

})

predictions = model.predict(new_students)

new_students["Prediction"] = np.where(
    predictions == 1,
    "PASS",
    "FAIL"
)

print(new_students)

```

	Study_Hours	Attendance	Previous_Marks	Assignment_Score	Prediction
0	6	85	72	80	PASS
1	2	60	45	50	FAIL
2	8	92	85	90	PASS

```

for i, prediction in enumerate(predictions):
    print(
        f"Student {i+1}: "
        f"{'PASS' if prediction == 1 else 'FAIL'}"
    )

```

Student 1: PASS
 Student 2: FAIL
 Student 3: PASS

```

from sklearn.metrics import confusion_matrix, classification_report

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Confusion Matrix:

```
[[2 0]
 [0 4]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	2
1	1.00	1.00	1.00	4
accuracy			1.00	6
macro avg	1.00	1.00	1.00	6
weighted avg	1.00	1.00	1.00	6



TASK 5 — 200–300 Word Report

Student Performance Prediction — Mini Project

This mini project focuses on predicting student performance using Machine Learning. A student performance dataset was created containing information about Study Hours, Attendance, Previous Marks,

and Assignment Score. These variables were selected because they can provide useful information about a student's academic performance. The target variable in the dataset is Result, which contains two classes: Pass and Fail.

The dataset was loaded using the Pandas library and checked for missing values and data types. Any missing records were removed during the data-cleaning stage. The categorical target variable was converted into numerical values, where Fail was represented by 0 and Pass was represented by 1. The dataset was then divided into training and testing sets using an 80:20 ratio.

A Decision Tree Classifier from Scikit-learn was used to train the model. The model learned relationships between the selected student features and their final result. After training, predictions were generated using the testing dataset, and model accuracy was calculated using the `accuracy_score()` function.

The model was also tested with three new student inputs. A student with 6 study hours, 85% attendance, previous marks of 72, and an assignment score of 80 was predicted as PASS. Another student with lower academic indicators was predicted as FAIL.

The main limitation is that the dataset is small and may not represent real student behavior accurately. In the future, the project could be improved by using a larger real-world dataset, adding more features such as exam preparation, project performance, and participation, and comparing multiple classification algorithms.



✓ Complete Mini-Project Code

```
# =====
# STUDENT PERFORMANCE PREDICTION
# =====

import pandas as pd
import numpy as np
import io

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix, classification_report

# Define the raw CSV data as a string
csv_data = """
Study_Hours,Attendance,Previous_Marks,Assignment_Score,Result
8,95,88,92,Pass
6,85,72,80,Pass
7,90,80,85,Pass
3,65,50,55,Fail
2,60,45,50,Fail
9,96,92,95,Pass
5,78,65,70,Pass
4,70,55,60,Fail
10,98,95,96,Pass
1,55,40,45,Fail
6,88,75,82,Pass
3,62,48,52,Fail
8,92,85,90,Pass
5,75,60,68,Pass
2,58,42,48,Fail
"""
```

```
7,86,78,84,Pass
4,68,52,58,Fail
9,94,90,93,Pass
6,82,70,76,Pass
3,64,46,54,Fail
8,89,82,87,Pass
2,57,44,46,Fail
5,80,68,72,Pass
4,72,58,62,Fail
10,97,94,98,Pass
7,91,81,86,Pass
1,52,35,40,Fail
6,84,73,79,Pass
3,66,49,56,Fail
9,93,89,91,Pass
"""

# 1. Load Dataset
df = pd.read_csv(io.StringIO(csv_data))

print("First 5 rows:")
print(df.head())

# 2. Dataset Information
print("\nDataset Shape:")
print(df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

# 3. Clean Dataset
df = df.dropna()

# 4. Convert Target
df["Result"] = df["Result"].map({
    "Fail": 0,
    "Pass": 1
})

# 5. Select Features
X = df[
    [
        "Study_Hours",
        "Attendance",
        "Previous_Marks",
        "Assignment_Score"
    ]
]

# Target
y = df["Result"]

# 6. Train-Test Split
```



```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# 7. Create Model
model = DecisionTreeClassifier(
    random_state=42,
    max_depth=4
)

# 8. Train Model
model.fit(X_train, y_train)

# 9. Make Predictions
y_pred = model.predict(X_test)

print("\nActual Values:")
print(y_test.values)

print("\nPredicted Values:")
print(y_pred)

# 10. Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")

# 11. Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# 12. Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# 13. Test New Students
new_students = pd.DataFrame({
    "Study_Hours": [6, 2, 8],
    "Attendance": [85, 60, 92],
    "Previous_Marks": [72, 45, 85],
    "Assignment_Score": [80, 50, 90]
})
```



```
predictions = model.predict(new_students)
```

```
# 14. Display Predictions
new_students["Prediction"] = np.where(
    predictions == 1,
    "PASS",
    "FAIL"
)
```

```
print("\nNew Student Predictions:")
print(new_students)
```

	Study_Hours	Attendance	Previous_Marks	Assignment_Score	Result
0	8	95	88	92	Pass
1	6	85	72	80	Pass
2	7	90	80	85	Pass
3	3	65	50	55	Fail
4	2	60	45	50	Fail

Dataset Shape:
(30, 5)

Data Types:

Study_Hours	int64
Attendance	int64
Previous_Marks	int64
Assignment_Score	int64
Result	object

dtype: object

Missing Values:

Study_Hours	0
Attendance	0
Previous_Marks	0
Assignment_Score	0
Result	0

dtype: int64

Training Data: (24, 4)
Testing Data: (6, 4)

Actual Values:
[0 1 1 1 1 0]

Predicted Values:
[0 1 1 1 1 0]

Model Accuracy: 1.0
Accuracy Percentage: 100.0 %

Confusion Matrix:
[[2 0]
[0 4]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	2
1	1.00	1.00	1.00	4
accuracy			1.00	6
macro avg	1.00	1.00	1.00	6
weighted avg	1.00	1.00	1.00	6



New Student Predictions:

	Study_Hours	Attendance	Previous_Marks	Assignment_Score	Prediction
0	6	85	72	80	PASS
1	2	60	45	50	FAIL
2	8	92	85	90	PASS

Start coding or [generate](#) with AI.

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.

